After exploring how the MVC design pattern applies to web API controllers, you'll see how a related topic works with web APIs: routing. We'll look at how explicit attribute routing works with action methods, touching on many of the same concepts we covered in chapters 6 and 14.

One of the big features added in ASP.NET Core 2.1 was the `[ApiController]` attribute. This attribute applies several common conventions used in web APIs, reducing the amount of code you must write yourself. In section 20.5 you'll learn how automatic `400 Bad Request`s for invalid requests, model-binding parameter inference, and `ProblemDetails` support make building APIs easier and more consistent.

You'll also learn how to format the API models returned by your action methods using content negotiation, to ensure that you generate a response that the calling client can understand. As part of this, you'll learn how to add support for additional format types, such as Extensible Markup Language (XML), so that you can generate XML responses if the client requests it.

Finally, I discuss some of the differences between API controllers and minimal API applications, and when you should choose one over the other. Before we get to that, we look at how to get started. In section 20.1 you'll see how to create a web API project and add your first API controller.

20.1 Creating your first web API project

In this section you'll learn how to create an ASP.NET Core web API project and create your first web API controllers. You'll see how to use controller action methods to handle HTTP requests and how to use `ActionResult`s to generate a response.